Request Desk: current solution and next phase

Codebase review: **28 October 2026**. The local assessment is implemented. The selected standalone platform is planned. Customer UAT and production acceptance remain pending.

Request Desk brings configurable forms, saved requests and review history into one React application. The legacy IT template covers equipment/software requests; blank forms support other synthetic processes. Business value should be measured through completion, missing-information returns and time to first review against an agreed baseline. No productivity savings have been measured.

Delivered locally

- Session login, scrypt passwords, CSRF checks, forced password change, account reset and session revocation.

- Admin account/form management and server-enforced create/view/review grants per form. Admin read access does not imply create/review rights.

- Blank/template authoring and a validated two-step Form.io JSON import flow, followed by editing/preview, immutable publication and backend-issued stable field identities. The importer supports a defined subset and rejects unsupported components, executable rules and remote data.

- Paginated request table with browser-history navigation, row actions and eligible inline review actions.

- Seven basic inputs plus multi-select, objects, repeating groups/tables and nested AND/OR visibility with root/row/parent scopes.

- Draft save/reload, validated submission, local attachment bytes and authorized preview/download, approval/rejection and audit.

- Confirmed draft upgrades, exact historical snapshot restoration and optimistic revision conflicts.

- Shared UI controls, admin modal dialogs, EN/VI switching and scoped Bootstrap/Form.io styles alongside Tailwind.

Local sessions, JSON records and file storage are real implementations using synthetic data. They do not provide production SSO, database transactions across stores, malware scanning or operational retention.

Architecture and historical data

React owns product state, locale and HTTP operations. **DynamicForm / FormHandle** isolates renderer lifecycle. Form.io renders legacy/basic schemas; React structured fields render array/object fields or custom visibility. Shared declarative functions implement custom visibility and API validation. The API owns identity, permissions, revision, status and persistence.

Authoring and publication

Blank forms require at least one field. The JSON importer validates input before replacing editor content and supports only a documented subset of Form.io components. The editor hides technical keys; draft save replaces temporary references with stable backend field IDs and rewrites conditions. Renaming or reordering preserves existing IDs. Published definitions remain immutable. Publishing advances the active version for new requests; saved requests load their pinned version.

Supported fields, labels, constraints and conditions can change through JSON without a frontend release. New input kinds, rule operators and domain integrations require code. The generated Form.io definition alone does not express all custom visibility semantics. Retain the authoritative fields configuration and compatibility tests.

Upgrade and restore

A draft owner previews an upgrade, inspects schema/data differences and corrects every invalid or missing required value. Confirmation checks the current revision, active target version and candidate fingerprint. The server recomputes the candidate before writing record, snapshots and audit through one atomic request-store replacement.

Restore recreates exact historical data, files and form version as a new revision. Missing historical schema/files or stale previews fail without changing the request. Submitted/final records cannot upgrade or restore. Review decisions require another owner's submitted/in-review request; rejection needs a reason and final decisions remain locked. There is no correction/resubmission workflow yet.

Operational limits

JSON persistence, sessions and throttling are single-process. Attachment bytes and form/request stores do not share a transaction. All snapshots remain local; there is no automatic retention. The large Form.io bundle still needs product performance budgets. Keyboard/mobile regression tests do not establish full accessibility conformance.

Selected next phase: standalone platform

The selected direction is a React-owned schema engine with PostgreSQL, S3-compatible object storage, OIDC and independent tenants from the first deployment. Use a modular monolith plus a durable worker. This replaces the earlier assumption that an existing product would supply every backend service. **G0-G9 remain planned, not delivered.**

Delivery sequence and gates

- G0-G1: establish baseline/contracts and unify engine semantics while preserving legacy schemas and historical behavior.

- G2-G4: transactional persistence, shared sessions, tenant membership/policies and controlled publication with approval, activation and rollback.

- G5-G7: typed rules/calculations/lookups, versioned workflow with correction and multiple approvals, reliable outbox actions and notifications.

- G8-G9: scanning/object lifecycle, snapshot-aware retention, independent audit export, operational telemetry, performance and acceptance evidence.

Each stage needs bounded changes, tests and review. The first production pilot must satisfy tenant isolation, verified migration, scan gates and backup/restore drills. Provider choice, target load, retention and RPO/RTO still need owners and decisions. An outbox provides retryable delivery; downstream idempotency/reconciliation is still required.

Estimate and team ownership

The earlier **four-week / 46 person-day** estimate applied to a narrow integration into an existing product: 38 base days and eight contingency. It is historical reference, not a commitment for this platform. Estimate G0 and then each dependent stage using confirmed scope and provider contracts. No platform completion date is committed.

The solution engineer owns boundaries and evidence; frontend engineers own authoring/rendering; backend engineers own tenant policy, transactions and delivery; QA owns regression and recovery tests. Business users own acceptance. Use small reviewed changes and stage gates, with bounded junior tasks and paired integration work.

Evidence, demonstration and acceptance

The latest local run passed **93 unit/API tests across 25 files** and **19 headless Chromium scenarios**. TypeScript and the Vite production build passed. Lint/format results and exact reproduction commands are recorded in docs/handoff.md. Hosted CI status, headed browser execution, customer UAT and production integrations were not verified by those runs.

Bounded schema-only release evidence

The integrated browser scenario publishes a new supported field, discovers it in the same requester document, preserves a v1 draft, confirms a valid upgrade, submits and approves through a separate reviewer. Before/after SHA-256 manifests and served-asset hashes match. This demonstrates the tested JSON change without a frontend build/deployment; it does not establish that arbitrary future features need no release.

Demonstration and handoff

Show JSON import or blank-form authoring and grants, requester save/reload and file preview, a new publication, explicit upgrade/restore and final review. Use the v6 presentation and Vietnamese speaker notes, then the live-demo script. The source includes tests, lockfile, CI workflow, contracts, architecture, operational runbook and the planned platform roadmap. The GitHub remote is configured; a workflow file alone does not prove a hosted run succeeded.

Acceptance and remaining risks

Local checks cover access boundaries, schema safety, persistence, upgrades/restores, retry/conflicts, JSON import, request pagination and desktop/mobile UI behavior. JavaScript is 2,249.39 kB minified / 655.04 kB gzip; CSS is 388.35 kB / 48.28 kB gzip. A chunk-size warning remains. These sizes are observations, not accepted performance budgets.

Customer UAT requires agreed business rules, representative users, a versioned sandbox and evidence for every critical journey. No critical/high defects may remain; accepted lesser issues need owners/dates, and a named business owner signs the exact scope/build. Platform UAT additionally needs tenant isolation, migration recovery, scan controls and reliable workflow delivery. Full cross-browser, screen-reader, load and real provider checks remain pending.

OpenAI Codex drafted code, tests and documentation and ran the recorded checks. Independent candidate review/rehearsal and business sign-off are not recorded. Review the implementation and explain its trade-offs in your own words. No real customer data or credentials belong in this package.